

EXPERIMENT 1.a LINEAR SEARCH

```
#include <stdio.h>
// Function to perform linear search
void linearSearch(int arr[], int size, int target, int result[], int
*resultSize) {
    *resultSize = 0; // Initialize the resultSize to 0

    // Iterate through the array
    for (int i = 0; i < size; i++) {
        // Check if the current element matches the target
        if (arr[i] == target) {
            // Add the matching index to the result array
            result[*resultSize] = i;
            (*resultSize)++; // Increment the resultSize
        }
    }
}

int main() {
    int size, target;
    // Input the size of the array
    printf("Enter the size of the array: ");
    scanf("%d", &size);
    int arr[size];
    // Input the elements of the array
    printf("Enter the elements of the array:\n");
    for (int i = 0; i < size; i++) {
        scanf("%d", &arr[i]);
    }
    // Input the target value to search
    printf("Enter the target value to search: ");
    scanf("%d", &target);
    // Declare an array to store matching indices
    int result[size];
    int resultSize;
    // Perform linear search
    linearSearch(arr, size, target, result, &resultSize);
    // Print the matching indices
    if (resultSize > 0)
    {
        printf("Matching indices for target %d: ", target);
        for (int i = 0; i < resultSize; i++) {
            printf("%d ", result[i]);
        }
        printf("\n");
    } else {
        printf("No matching indices found for target %d.\n", target);
    }
    return 0;
}
```

EXPERIMENT 1.B BINARY SEARCH

```
#include <stdio.h>
// Function to validate if the array is sorted in ascending order
int validateSortedArray(int arr[], int size) {
    for (int i = 0; i < size - 1; ++i) {
        if (arr[i] > arr[i + 1]) {
            return 0; // Not sorted
        }
    }
    return 1; // Sorted
}
// Function to perform binary search on a sorted list of integers
int binarySearch(int arr[], int size, int target) {
    int low = 0;
    int high = size - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == target) {
            return mid; // Target value found, return the index
        } else if (arr[mid] < target) {
            low = mid + 1; // Search the right half
        } else {
            high = mid - 1; // Search the left half
        }
    }
    return -1; // Target value not found
}
int main() {
    int n, target;
    printf("Enter the number of elements: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter the sorted elements:\n");
    for (int i = 0; i < n; ++i) {
        scanf("%d", &arr[i]);
    }
    // Validate if the array is sorted
    if (validateSortedArray(arr, n)) {
        printf("Enter the element to be searched: ");
        scanf("%d", &target);
        // Perform binary search
        int result = binarySearch(arr, n, target);
        if (result != -1) {
            printf("Target value %d found at index: %d\n", target,
result);
        } else {
            printf("Target value %d not found in the array.\n",
target);
        }
    }
}
```

```

    } else {
        printf("The entered array is not sorted.\n");
    }

    return 0;
}

```

EX. NO. 2 MERGE SORT

```

#include <stdio.h>
#include <stdlib.h>
// Function to merge two sorted halves of an array
void merge(int arr[], int left, int mid, int right)
{
    int i, j, k;
    int n1 = mid - left + 1;
    int n2 = right - mid;

    // Create temporary arrays
    int L[n1], R[n2];

    // Copy data to temporary arrays L[] and R[]
    for (i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    // Merge the temporary arrays back into arr[left..right]
    i = 0;
    j = 0;
    k = left;

    while (i < n1 && j < n2)
    {
        if (L[i] <= R[j])
        {
            arr[k] = L[i];
            i++;
        }
        else
        {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    // Copy the remaining elements of L[], if there are any
    while (i < n1)

```

```

    {
        arr[k] = L[i];
        i++;
        k++;
    }

    // Copy the remaining elements of R[], if there are any
    while (j < n2)
    {
        arr[k] = R[j];
        j++;
        k++;
    }
}

// Function to perform Merge Sort
void mergeSort(int arr[], int left, int right)
{
    if (left < right)
    {
        // Same as (left+right)/2, but avoids overflow for large left and right
        int mid = left + (right - left) / 2;

        // Recursively sort the first and second halves
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        // Merge the sorted halves
        merge(arr, left, mid, right);
    }
}

// Function to print an array
void printArray(int arr[], int size)
{
    for (int i = 0; i < size; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

int main()
{
    int n;

    printf("Enter the number of elements: ");
    scanf("%d", &n);

    int arr[n];

```

```

printf("Enter the elements:\n");
for (int i = 0; i < n; i++)
{
    scanf("%d", &arr[i]);
}

// Perform Merge Sort
mergeSort(arr, 0, n - 1);

// Print the sorted array
printf("Sorted array: ");
printArray(arr, n);

return 0;
}

```

EX. NO.3 DFS

```

#include <stdio.h>
#include <stdbool.h>

#define MAX_VERTICES 100

// Function to initialize the graph
void initializeGraph(int vertices, int adjacencyMatrix[MAX_VERTICES][MAX_VERTICES])
{
    // Initialize the adjacency matrix with zeros
    for (int i = 0; i < vertices; ++i)
    {
        for (int j = 0; j < vertices; ++j)
        {
            adjacencyMatrix[i][j] = 0;
        }
    }
}

// Function to add an edge to the graph
void addEdge(int adjacencyMatrix[MAX_VERTICES][MAX_VERTICES], int start, int end)
{
    adjacencyMatrix[start][end] = 1;
    adjacencyMatrix[end][start] = 1; // For undirected graph
}

// Depth-First Search (DFS) function
void DFS(int vertices, int adjacencyMatrix[MAX_VERTICES][MAX_VERTICES], int vertex, bool visited[])
{

```

```

visited[vertex] = true;

for (int i = 0; i < vertices; ++i)
{
    if (adjacencyMatrix[vertex][i] && !visited[i])
    {
        DFS(vertices, adjacencyMatrix, i, visited);
    }
}
}

// Function to check if the graph is connected using DFS
bool isConnected(int vertices, int adjacencyMatrix[MAX_VERTICES][MAX_VERTICES])
{
    bool visited[MAX_VERTICES] = {false};

    // Start DFS from the first vertex
    DFS(vertices, adjacencyMatrix, 0, visited);

    // Check if all vertices are visited
    for (int i = 0; i < vertices; ++i)
    {
        if (!visited[i])
        {
            return false; // Graph is not connected
        }
    }

    return true; // Graph is connected
}

int main()
{
    int vertices, edges, start, end;
    int adjacencyMatrix[MAX_VERTICES][MAX_VERTICES];

    printf("Enter the number of vertices in the graph: ");
    scanf("%d", &vertices);

    initializeGraph(vertices, adjacencyMatrix);

    printf("Enter the number of edges in the graph: ");
    scanf("%d", &edges);

```

```

printf("Enter the edges (start end):\n");
printf("Use starting vertex ID as 0 \n");
for (int i = 0; i < edges; ++i)
{
    scanf("%d %d", &start, &end);
    addEdge(adjacencyMatrix, start, end);
}

if (isConnected(vertices, adjacencyMatrix))
{
    printf("The graph is connected.\n");
}
else
{
    printf("The graph is not connected.\n");
}

return 0;
}

```

Ex. No. 4 BFS

```

#include <stdio.h>
#include <stdlib.h>
#define MAX_VERTICES 100
int queue[MAX_VERTICES];
int front = -1, rear = -1;
// Function to enqueue an element into the queue
void enqueue(int data) {
    // Check for queue overflow
    if (rear == MAX_VERTICES - 1) {
        printf("Queue Overflow\n");
        return;
    }
    // If queue is empty, set front to 0
    if (front == -1) {
        front = 0;
    }
    // Enqueue the data
    queue[++rear] = data;
}
// Function to dequeue an element from the queue
int dequeue() {
    // Check for queue underflow
    if (front == -1 || front > rear) {
        printf("Queue Underflow\n");
    }
}

```

```

        return -1;
    }
    // Dequeue the element and return
    return queue[front++];
}
// Function to check if the queue is empty
int isEmpty()
{
    return front == -1 || front > rear;
}
// Breadth-First Search traversal function
void bfs(int graph[MAX_VERTICES][MAX_VERTICES], int numVertices, int startVertex, int visited[]) {
    // Mark the starting vertex as visited and enqueue it
    visited[startVertex] = 1;
    enqueue(startVertex);
    // Perform BFS traversal
    while (!isEmpty()) {
        int currentVertex = dequeue();
        printf("%d ", currentVertex);
        // Visit adjacent vertices
        for (int i = 0; i < numVertices; i++) {
            if (graph[currentVertex][i] == 1 && !visited[i]) {
                visited[i] = 1;
                enqueue(i);
            }
        }
    }
}
int main() {
    int numVertices, graph[MAX_VERTICES][MAX_VERTICES], visited[MAX_VERTICES];
    // Input the number of vertices
    printf("Enter the number of vertices: ");
    scanf("%d", &numVertices);
    // Input the adjacency matrix
    printf("Enter the adjacency matrix:\n");
    for (int i = 0; i < numVertices; i++) {
        for (int j = 0; j < numVertices; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    // Input the starting vertex for BFS
    printf("Enter the starting vertex for BFS: ");
    int startVertex;

```



```

scanf("%d", &startVertex);
// Initialize the visited array
for (int i = 0; i < numVertices; i++) {
    visited[i] = 0;
}
// Perform BFS traversal and display the result
printf("BFS Traversal: ");
bfs(graph, numVertices, startVertex, visited);
return 0;
}

```

EX. NO.5 WARSHALL'S ALGORITHM

```

//Only for faculty reference, do not share the code with students
#include <stdio.h>
// Function to compute transitive closure using Warshall's algorithm
void transitiveClosure(int graph[][100], int n)
{
    int i, j, k;
    // Adding vertices individually
    for (k = 0; k < n; k++)
    {
        // Fixing source vertices one by one
        for (i = 0; i < n; i++)
        {
            // Fixing destination vertices one by one
            for (j = 0; j < n; j++)
            {
                // If vertex k is on a path from i to j,
                // then mark it as reachable
                if (graph[i][k] && graph[k][j])
                    graph[i][j] = 1;
            }
        }
    }
}

// Displaying the transitive closure
printf("Transitive closure of the given graph is:\n");
for (i = 0; i < n; i++)
{
    for (j = 0; j < n; j++)
    {
        printf("%d ", graph[i][j]);
    }
    printf("\n");
}
}

```

```

int main() {
    int graph[100][100], n, i, j;

    printf("Enter the number of vertices: ");
    scanf("%d", &n);

    // Input adjacency matrix
    printf("Enter the adjacency matrix (space-separated):\n");
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            scanf("%d", &graph[i][j]);
        }
    }

    // Compute transitive closure
    transitiveClosure(graph, n);

    return 0;
}

```

EX.NO.6 FLOYD'S ALGORITHM

```

#include <stdio.h>

#define INF 99999

#define V 4

void printSolution(int dist[][V]);
void floydWarshall(int graph[][V]) {
    int dist[V][V];

    int i, j, k;

    // Initialize the solution matrix same as input graph matrix
    for (i = 0; i < V; i++)
        for (j = 0; j < V; j++)
            dist[i][j] = graph[i][j];

    // Add all vertices one by one to the set of intermediate vertices
    for (k = 0; k < V; k++) {
        // Pick all vertices as source one by one

```

```

for (i = 0; i < V; i++) {
    // Pick all vertices as destination for the
    // above picked source
    for (j = 0; j < V; j++) {
        // If vertex k is on the shortest path from
        // i to j, then update the value of dist[i][j]
        if (dist[i][k] + dist[k][j] < dist[i][j])
            dist[i][j] = dist[i][k] + dist[k][j];
    }
}

// Print the shortest distance matrix
printSolution(dist);
}

void printSolution(int dist[][V]) {
    printf ("The following matrix shows the shortest distances between every pair of vertices:\n");
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            if (dist[i][j] == INF)
                printf("%7s", "INF");
            else
                printf("%7d", dist[i][j]);
        }
        printf("\n");
    }
}

```

```

int main() {
    int graph[V][V] = {{0, 3, INF, 7},
                        {8, 0, 2, INF},
                        {5, INF, 0, 1},
                        {2, INF, INF, 0}
                       };

    floydWarshall(graph);

    return 0;
}

```

EX.NO.7 WARSHALL'S ALGORITHM

```

#include<stdio.h>

// Function to find maximum of two integers
int max(int a, int b) {
    return (a > b) ? a : b;
}

// Function to solve 0/1 Knapsack problem using Dynamic Programming
int knapsack(int W, int wt[], int val[], int n)
{
    int i, w;

    int K[n + 1][W + 1];

    // Build K[][] table
    for (i = 0; i <= n; i++)
    {
        for (w = 0; w <= W; w++)
        {
            if (i == 0 || w == 0)
                K[i][w] = 0;
            else if (wt[i - 1] <= w)

```

```

        K[i][w] = max(val[i - 1] + K[i - 1][w - wt[i - 1]], K[i - 1][w]);
    else
        K[i][w] = K[i - 1][w];
    }
}

return K[n][W];
}

// Main function
int main()
{
    int val[] = {12, 10, 20, 15};
    int wt[] = {2, 1, 3, 2};
    int W = 5;
    int n = sizeof(val) / sizeof(val[0]);
    printf("Maximum value that can be obtained is %d\n", knapsack(W, wt, val, n));
    return 0;
}

```

EX. NO. 8 MINIMUM SPANNING TREE

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// Structure to represent an edge in the graph
```

```

struct Edge {
    int src, dest, weight;
};

```

```
// Structure to represent a subset for union-find
```

```

struct Subset {
    int parent;
    int rank;
};

// Function prototypes
int find(struct Subset subsets[], int i);
void Union(struct Subset subsets[], int x, int y);
int compareEdges(const void* a, const void* b);
void KruskalMST(struct Edge edges[], int V, int E);

int main() {
    int V, E;
    printf("Enter the number of vertices and edges: ");
    scanf("%d %d", &V, &E);

    struct Edge* edges = (struct Edge*)malloc(E * sizeof(struct Edge));
    printf("Enter source, destination, and weight for each edge:\n");
    for (int i = 0; i < E; ++i)
        scanf("%d %d %d", &edges[i].src, &edges[i].dest, &edges[i].weight);

    KruskalMST(edges, V, E);

    free(edges);
    return 0;
}

// Find set of an element i (uses path compression technique)
int find(struct Subset subsets[], int i) {

```

```

    if (subsets[i].parent != i)
        subsets[i].parent = find(subsets, subsets[i].parent);
    return subsets[i].parent;
}

```

// Union of two sets of x and y (uses union by rank)

```

void Union(struct Subset subsets[], int x, int y) {
    int xroot = find(subsets, x);
    int yroot = find(subsets, y);

    if (subsets[xroot].rank < subsets[yroot].rank)
        subsets[xroot].parent = yroot;
    else if (subsets[xroot].rank > subsets[yroot].rank)
        subsets[yroot].parent = xroot;
    else {
        subsets[yroot].parent = xroot;
        subsets[xroot].rank++;
    }
}

```

// Comparator function for sorting edges based on weight

```

int compareEdges(const void* a, const void* b) {
    struct Edge* edge1 = (struct Edge*)a;
    struct Edge* edge2 = (struct Edge*)b;
    return edge1->weight - edge2->weight;
}

```

// Kruskal's algorithm for finding Minimum Spanning Tree

```

void KruskalMST(struct Edge edges[], int V, int E) {

```

```

qsort(edges, E, sizeof(edges[0]), compareEdges);

struct Subset* subsets = (struct Subset*)malloc(V * sizeof(struct Subset));

for (int v = 0; v < V; ++v) {
    subsets[v].parent = v;
    subsets[v].rank = 0;
}

struct Edge* result = (struct Edge*)malloc((V - 1) * sizeof(struct Edge));
int e = 0; // Index for result[]

int i = 0; // Index for sorted edges[]
while (e < V - 1 && i < E) {
    struct Edge next_edge = edges[i++];

    int x = find(subsets, next_edge.src);
    int y = find(subsets, next_edge.dest);

    if (x != y) {
        result[e++] = next_edge;
        Union(subsets, x, y);
    }
}

printf("Edges in the Minimum Spanning Tree:\n");
for (int j = 0; j < e; ++j)
    printf("%d -- %d == %d\n", result[j].src, result[j].dest, result[j].weight);

```



```
    free(subsets);  
    free(result);  
}
```

EX.NO.9 DIJKSTRA'S ALGORITHM

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <limits.h>
```

```
#define V 9
```

```
int minDistance(int dist[], int sptSet[])
```

```
{  
    int min = INT_MAX, min_index;  
  
    for (int v = 0; v < V; v++)  
        if (sptSet[v] == 0 && dist[v] <= min)  
            min = dist[v], min_index = v;  
  
    return min_index;  
}
```

```
void printSolution(int dist[], int n)
```

```
{  
    printf("Vertex \t Distance from Source\n");  
    for (int i = 0; i < V; i++)  
        printf("%d \t %d\n", i, dist[i]);  
}
```

```
void dijkstra(int graph[V][V], int src)
```

```

{
    int dist[V];

    int sptSet[V];

    for (int i = 0; i < V; i++)
        dist[i] = INT_MAX, sptSet[i] = 0;

    dist[src] = 0;

    for (int count = 0; count < V - 1; count++) {
        int u = minDistance(dist, sptSet);
        sptSet[u] = 1;

        for (int v = 0; v < V; v++)
            if (!sptSet[v] && graph[u][v] && dist[u] != INT_MAX && dist[u] + graph[u][v] < dist[v])
                dist[v] = dist[u] + graph[u][v];
    }
    printSolution(dist, V);
}

```

```

int main()
{
    int graph[V][V] = {{0, 4, 0, 0, 0, 0, 0, 8, 0},
                        {4, 0, 8, 0, 0, 0, 0, 11, 0},
                        {0, 8, 0, 7, 0, 4, 0, 0, 2},
                        {0, 0, 7, 0, 9, 14, 0, 0, 0},
                        {0, 0, 0, 9, 0, 10, 0, 0, 0},
                        {0, 0, 4, 14, 10, 0, 2, 0, 0},
                        {0, 0, 0, 0, 0, 2, 0, 1, 6},

```

```
{8, 11, 0, 0, 0, 0, 1, 0, 7},
```

```
{0, 0, 2, 0, 0, 0, 6, 7, 0}};
```

```
dijkstra(graph, 0);
```

```
return 0;
```

```
}
```